FreeRTOS、CMSIS-RTOS2 与 STM32 HAL 基础及工程实践

1. 文档说明

本文结合用户提供的 CSDN 文章主题，整理 FreeRTOS 的基本原理，并补充：

- CMSIS-RTOS2 常用 API
- FreeRTOS 原生 API
- STM32 HAL 外设 API
- 任务、栈、队列、信号量、互斥锁、临界区
- 中断与任务之间的通信
- 软件定时器、事件标志组、任务通知、内存池
- 在当前 STM32 平衡车项目中的任务划分建议

用户提供的文章链接：

https://blog.csdn.net/sjwnghuoyuelaiyu/article/details/162584054

说明：该链接正文无法稳定抓取，因此本文不逐字复制文章内容，而是结合文章涉及的 FreeRTOS 学习主题和官方 API 文档进行独立整理。

---

2. FreeRTOS 是什么

FreeRTOS 是面向微控制器的实时操作系统内核。它主要解决的问题是：

```
如何让多个具有不同周期和优先级的功能
在一个 CPU 上有条理地运行
```

在裸机程序中，常见结构是：

```
while (1)
{
    按键扫描();
    读取传感器();
    更新显示();
    处理蓝牙();
}
```

当功能增多以后，容易出现：

- 某个函数阻塞，导致其他功能得不到执行
- 延时函数影响整个主循环
- 中断中放入过多业务逻辑
- 不同周期的任务互相影响
- 全局变量读写关系越来越复杂

使用 RTOS 后，可以把程序拆成多个任务：

```
角度控制任务     100 Hz
速度控制任务      50 Hz
蓝牙处理任务      20 Hz
```

```
OLED 显示任务     10 Hz
按键处理任务      20 Hz
日志输出任务       5 Hz
```

每个任务拥有独立的函数、栈和优先级，由调度器负责安排运行。

---

3. RTOS 的核心概念

3.1 任务

任务可以理解为一个独立运行的函数：

```
void AngleTask(void *argument)
{
  for (;;)
  {
    MPU6050_Update();
    Angle_Control();
    osDelay(10);
  }
}
```

任务函数通常不能正常返回，而是长期运行在无限循环中。

3.2 调度器

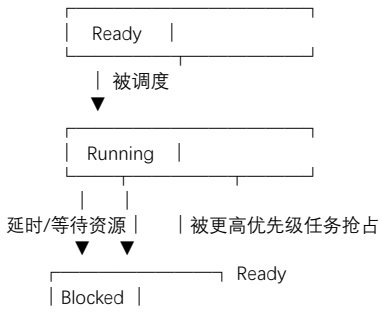
调度器根据以下条件决定运行哪个任务：

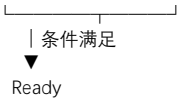
1. 任务是否处于就绪状态
2. 任务优先级
3. 是否启用抢占
4. 是否启用时间片轮转
5. 任务是否因延时或等待资源而阻塞

简化后的调度过程：

```
所有任务
  ↓
筛选处于 Ready 状态的任务
  ↓
选择优先级最高的任务
  ↓
运行该任务
  ↓
任务延时、等待消息或被抢占
  ↓
重新调度
```

3.3 任务状态





Suspended：被显式挂起，不会参与调度

3.4 系统节拍 Tick

RTOS 通常使用一个周期性中断作为系统节拍：

```
Tick 频率 = configTICK_RATE_HZ
```

例如：

```
configTICK_RATE_HZ = 1000
```

表示每 1 ms 产生一个 RTOS Tick。

Tick 用于：

- 任务延时
- 超时判断
- 软件定时器
- 时间片调度
- 阻塞等待

在 STM32 工程中要注意：FreeRTOS 和 HAL 都可能使用 SysTick。应统一规划时间基准，避免两个系统同时错误地接管 SysTick。

---

4. FreeRTOS 与 CMSIS-RTOS2 的关系

4.1 两层结构

```
应用程序
  ↓
CMSIS-RTOS2 API
  ↓
FreeRTOS 内核适配层
  ↓
FreeRTOS Kernel
  ↓
STM32 Cortex-M 硬件
```

CMSIS-RTOS2 是一套统一 API。应用代码调用 osThreadNew()、osMessageQueuePut() 等函数，而底层可以由 FreeRTOS、RTX 等不同内核实现。

4.2 两种常见编程方式

CMSIS-RTOS2 写法：

```
#include "cmsis_os2.h"

osThreadId_t task_id;

void WorkerTask(void *argument)
{
   for (;;)
```

```
    {
        osDelay(100);
    }
}

void RTOS_Init(void)
{
    task_id = osThreadNew(WorkerTask, NULL, NULL);
}
```

FreeRTOS 原生写法：

```
#include "FreeRTOS.h"
#include "task.h"

TaskHandle_t task_handle;

void WorkerTask(void *argument)
{
    for (;;)
    {
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void RTOS_Init(void)
{
    xTaskCreate(WorkerTask,
                "Worker",
                256,
                NULL,
                2,
                &task_handle);
}
```

CMSIS-RTOS2 更便于跨 RTOS 移植；FreeRTOS 原生 API 功能更完整、资料更多。

---

5. CMSIS-RTOS2 常用 API 总表

功能	CMSIS-RTOS2 API
初始化内核	osKernelInitialize()
启动内核	osKernelStart()
创建任务	osThreadNew()
任务延时	osDelay()
周期延时	osDelayUntil()
挂起任务	osThreadSuspend()
恢复任务	osThreadResume()
删除任务	osThreadTerminate()
创建消息队列	osMessageQueueNew()
发送消息	osMessageQueuePut()
接收消息	osMessageQueueGet()
创建信号量	osSemaphoreNew()
获取信号量	osSemaphoreAcquire()
释放信号量	osSemaphoreRelease()
创建互斥锁	osMutexNew()
获取互斥锁	osMutexAcquire()

功能	CMSIS-RTOS2 API
释放互斥锁	osMutexRelease()
创建事件标志	osEventFlagsNew()
设置事件标志	osEventFlagsSet()
等待事件标志	osEventFlagsWait()
创建软件定时器	osTimerNew()
启动软件定时器	osTimerStart()
创建内存池	osMemoryPoolNew()
申请内存块	osMemoryPoolAlloc()
释放内存块	osMemoryPoolFree()
获取系统 Tick	osKernelGetTickCount()

---

6. 任务和任务栈

6.1 任务栈的作用

每个任务都需要独立栈空间，用来保存：

- 局部变量
- 函数参数
- 函数返回地址
- 中断或任务切换上下文
- 嵌套函数调用数据



栈太小可能导致：

- 程序随机死机
- HardFault
- 变量被覆盖
- 任务运行一段时间后异常

栈太大则浪费 RAM。

6.2 CMSIS-RTOS2 任务创建

```
const osThreadAttr_t SensorTask_attributes = {
  .name = "SensorTask",
  .priority = osPriorityAboveNormal,
  .stack_size = 512
};
```

```
osThreadId_t SensorTaskHandle;

SensorTaskHandle = osThreadNew(SensorTask,
                               NULL,
                               &SensorTask_attributes);
```

6.3 FreeRTOS 原生任务创建

```
xTaskCreate(SensorTask,
            "SensorTask",
            256,
            NULL,
            4,
            &SensorTaskHandle);
```

FreeRTOS 的栈深度单位通常是 StackType_t 数量，不一定是字节。CMSIS-RTOS2 的 stack_size 通常按字节表达，实际使用时应以具体实现和配置为准。

6.4 周期任务

不推荐用：

```
for (;;)
{
  Work();
  osDelay(10);
}
```

因为 Work() 执行时间会造成周期漂移。

更适合固定周期任务：

```
void ControlTask(void *argument)
{
  uint32_t next_tick = osKernelGetTickCount();

  for (;;)
  {
    next_tick += 10;
    Control_Update();
    osDelayUntil(next_tick);
  }
}
```

FreeRTOS 原生写法：

```
void ControlTask(void *argument)
{
  TickType_t last_wake = xTaskGetTickCount();

  for (;;)
  {
    Control_Update();
    vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(10));
  }
}
```

---

7. 队列 Queue

7.1 队列是什么

队列是任务之间传递数据的 FIFO 缓冲区：

```
生产者任务 —— Put ——> [消息1][消息2][消息3] —— Get ——> 消费者任务
```

队列适合传递：

- 传感器数据
- 串口数据帧
- 控制命令
- 事件结构体
- 多个待处理请求

7.2 CMSIS-RTOS2 队列

```
typedef struct
{
  int16_t speed;
  int16_t turn;
} ControlMessage_t;

osMessageQueueId_t control_queue;

control_queue = osMessageQueueNew(8,
                                  sizeof(ControlMessage_t),
                                  NULL);
```

发送：

```
ControlMessage_t msg = {
  .speed = 100,
  .turn = 0
};

osMessageQueuePut(control_queue,
                  &msg,
                  0,
                  0);
```

接收：

```
ControlMessage_t msg;

if (osMessageQueueGet(control_queue,
                      &msg,
                      NULL,
                      osWaitForever) == osOK)
{
  SetSpeed(msg.speed);
  SetTurn(msg.turn);
}
```

7.3 FreeRTOS 原生队列

```
QueueHandle_t control_queue;

control_queue = xQueueCreate(8,
                             sizeof(ControlMessage_t));
```

发送：

```
xQueueSend(control_queue,
           &msg,
           0);
```

接收：

```
xQueueReceive(control_queue,
```

```
        &msg,
        portMAX_DELAY);
```

7.4 中断发送队列

中断中不能使用普通阻塞 API，应使用 ISR 版本：

```
BaseType_t higher_priority_task_woken = pdFALSE;

xQueueSendFromISR(control_queue,
                  &msg,
                  &higher_priority_task_woken);

portYIELD_FROM_ISR(higher_priority_task_woken);
```

CMSIS-RTOS2 中，部分 osMessageQueuePut()、osMessageQueueGet() 操作支持 ISR，但 ISR 中必须使用 timeout = 0，不能阻塞等待。

7.5 队列的优缺点

优点：

- 传递数据安全
- 可以缓存多个消息
- 生产者和消费者解耦
- 支持任务到任务、ISR 到任务

缺点：

- 会复制消息内容
- 队列过大占用 RAM
- 消息处理不及时可能溢出
- 不适合无限制的数据流传输

---

8. 信号量 Semaphore

8.1 二值信号量

二值信号量只有两种状态：

```
有信号 / 无信号
```

常用于：

- 外部中断通知任务
- DMA 完成通知任务
- 数据采集完成通知任务

```
中断/DMA完成
    ↓ Release
二值信号量
    ↓ Acquire
处理任务被唤醒
```

CMSIS-RTOS2：

```
osSemaphoreId_t data_ready_sem;

data_ready_sem = osSemaphoreNew(1, 0, NULL);
```

中断或生产者：

```
osSemaphoreRelease(data_ready_sem);
```

消费者任务：

```
osSemaphoreAcquire(data_ready_sem,
                   osWaitForever);

Process_Data();
```

8.2 计数信号量

计数信号量的值可以大于 1，适合表示资源数量：

```
可用缓冲区数量
可用硬件资源数量
待处理事件数量
```

```
osSemaphoreId_t buffer_sem;

buffer_sem = osSemaphoreNew(4, 4, NULL);
```

每申请一个缓冲区：

```
osSemaphoreAcquire(buffer_sem, osWaitForever);
```

释放一个缓冲区：

```
osSemaphoreRelease(buffer_sem);
```

8.3 信号量和队列区别

项目	信号量	队列
是否传递数据	否	是
是否表示资源数量	可以	不适合
是否缓存多个数据	计数信号量只能计数	可以缓存消息
典型用途	事件通知	消息传递

---

9. 互斥锁 Mutex

9.1 互斥锁的作用

互斥锁用于保护共享资源，保证同一时刻只有一个任务进入临界代码：

```
任务 A —— Acquire ——> 使用 OLED —— Release
任务 B —— Acquire ——> 等待
```

典型共享资源：

- OLED
- UART 打印
- I2C 总线
- SPI 总线
- 文件系统
- 共享数据结构

9.2 CMSIS-RTOS2 使用

```
osMutexId_t oled_mutex;

oled_mutex = osMutexNew(NULL);


osMutexAcquire(oled_mutex, osWaitForever);

OLED_Clear();
OLED_Printf(0, 0, OLED_6X8, "Speed:%d", speed);
OLED_Update();

osMutexRelease(oled_mutex);
```

9.3 FreeRTOS 原生使用

```
SemaphoreHandle_t oled_mutex;

oled_mutex = xSemaphoreCreateMutex();


if (xSemaphoreTake(oled_mutex,
                   portMAX_DELAY) == pdTRUE)
{
    OLED_Update();
    xSemaphoreGive(oled_mutex);
}
```

9.4 互斥锁和二值信号量区别

项目	互斥锁	二值信号量
主要用途	保护共享资源	事件通知
是否有所有者	有	没有
是否支持优先级继承	通常支持	不用于资源所有权
是否适合 ISR 释放	不应在 ISR 中使用	可用于 ISR 通知
典型例子	保护 OLED	DMA 完成通知

9.5 优先级反转

可能出现：

```
低优先级任务持有锁
高优先级任务等待锁
中优先级任务不断运行
低优先级任务无法释放锁
高优先级任务反而长时间不能运行
```

互斥锁的优先级继承机制可以缓解这个问题，但不能替代合理的资源设计。

---

10. 临界区 Critical Section

10.1 临界区作用

临界区用于保护不能被打断的短小代码：

```
taskENTER_CRITICAL();

shared_value++;

taskEXIT_CRITICAL();
```

在 Cortex-M 上，临界区通常会屏蔽一定优先级范围内的中断。

10.2 CMSIS-RTOS2 中的注意事项

CMSIS-RTOS2 主要提供任务同步
API，不建议用它替代所有硬件临界区操作。对于极短的硬件寄存器或共享变量保护，可以使用 FreeRTOS 原生宏：

```
taskENTER_CRITICAL();
taskEXIT_CRITICAL();
```

10.3 ISR 临界区

在中断中应使用对应的 ISR 版本和正确的中断保护方式，不要在 ISR 中执行：

- osDelay()
- vTaskDelay()
- osMutexAcquire()
- xSemaphoreTake() 阻塞等待
- 阻塞式 HAL 通信
- OLED 刷屏

临界区必须尽可能短，否则会导致：

- 丢失串口数据
- 延迟其他中断
- RTOS 调度延迟
- 控制周期抖动

---

11. 任务通知 Task Notification

任务通知是 FreeRTOS 提供的轻量级任务间通信方式。

它通常比队列和信号量占用更少的 RAM，速度更快，但目标通常是特定任务。

11.1 任务通知示例

```
TaskHandle_t sensor_task_handle;
```

中断中通知：

```
BaseType_t higher_priority_task_woken = pdFALSE;

vTaskNotifyGiveFromISR(sensor_task_handle,
```

```
                    &higher_priority_task_woken);

portYIELD_FROM_ISR(higher_priority_task_woken);
```

任务中等待：

```
ulTaskNotifyTake(pdTRUE,
                 portMAX_DELAY);

Process_Sensor_Data();
```

适合：

- 一个中断通知一个任务
- DMA 完成通知
- 单事件唤醒
- 计数事件

不适合：

- 需要缓存多条复杂消息
- 多个消费者共同接收同一消息

---

12. 事件标志组 Event Flags

事件标志组可以用一个整数的不同 bit 表示多个事件：

```
bit0：MPU 数据准备完成
bit1：蓝牙数据准备完成
bit2：编码器数据准备完成
bit3：按键事件发生
```

CMSIS-RTOS2：

```
osEventFlagsId_t event_flags;

event_flags = osEventFlagsNew(NULL);
```

设置事件：

```
osEventFlagsSet(event_flags, 1U << 0);
```

等待事件：

```
uint32_t flags;

flags = osEventFlagsWait(event_flags,
                         (1U << 0) | (1U << 1),
                         osFlagsWaitAny,
                         osWaitForever);
```

适合：

- 多个状态事件
- 系统启动条件

- 多个外设初始化完成通知

---

13. 软件定时器

软件定时器由 RTOS 定时器服务任务统一管理，不需要为每个定时器占用一个任务。

CMSIS-RTOS2：

```
void ControlTimerCallback(void *argument)
{
  Control_Request_Update();
}

osTimerId_t control_timer;

control_timer = osTimerNew(ControlTimerCallback,
                           osTimerPeriodic,
                           NULL,
                           NULL);

osTimerStart(control_timer, 10);
```

注意：定时器回调运行在定时器服务任务中，不要在回调中执行长时间阻塞操作。

适合：

- 周期性发送心跳
- 软件超时
- 通信超时检测
- LED 闪烁

如果角度环是严格的实时闭环，不建议只依赖普通软件定时器，而应结合硬件定时器或高优先级周期任务。

---

14. 内存池 Memory Pool

内存池提供固定大小的数据块：

```
固定大小内存块1
固定大小内存块2
固定大小内存块3
```

它比通用动态内存更容易预测，适合：

- 固定大小消息
- 数据包缓冲区
- 传感器采样对象
- 网络或串口缓冲节点

CMSIS-RTOS2：

```
typedef struct
{
  uint32_t timestamp;
  float pitch;
  float speed;
```

```
} SensorSample_t;

osMemoryPoolId_t sample_pool;

sample_pool = osMemoryPoolNew(8,
                              sizeof(SensorSample_t),
                              NULL);
```

申请和释放：

```
SensorSample_t *sample;

sample = osMemoryPoolAlloc(sample_pool, 0);

if (sample != NULL)
{
   sample->pitch = current_pitch;
   osMemoryPoolFree(sample_pool, sample);
}
```

---

15. STM32 HAL 与 FreeRTOS 的关系

HAL 是硬件抽象层，负责操作 STM32 外设；FreeRTOS 负责任务调度和任务间同步。

```
FreeRTOS：
任务、队列、信号量、互斥锁、定时器

HAL：
GPIO、UART、I2C、SPI、TIM、ADC、DMA
```

15.1 HAL 常用初始化函数

```
HAL_Init();
SystemClock_Config();

MX_GPIO_Init();
MX_USART1_UART_Init();
MX_I2C1_Init();
MX_SPI1_Init();
MX_TIM1_Init();
```

常见 HAL 初始化流程：

```
HAL_Init()
   ↓
系统时钟
   ↓
GPIO、DMA、UART、I2C、SPI、TIM 初始化
   ↓
FreeRTOS 对象创建
   ↓
启动调度器
```

15.2 GPIO

```
HAL_GPIO_WritePin(GPIOA,
                  GPIO_PIN_0,
                  GPIO_PIN_SET);

GPIO_PinState state;

state = HAL_GPIO_ReadPin(GPIOA,
                         GPIO_PIN_0);

HAL_GPIO_TogglePin(GPIOA,
                   GPIO_PIN_0);
```

15.3 UART

阻塞发送：

```
HAL_UART_Transmit(&huart1,
                  data,
                  length,
                  100);
```

阻塞接收：

```
HAL_UART_Receive(&huart1,
                 data,
                 length,
                 100);
```

中断发送：

```
HAL_UART_Transmit_IT(&huart1,
                     data,
                     length);
```

中断接收：

```
HAL_UART_Receive_IT(&huart1,
                    &rx_byte,
                    1);
```

回调函数：

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
   if (huart->Instance == USART1)
   {
      Parse_Byte(rx_byte);

      HAL_UART_Receive_IT(&huart1,
                          &rx_byte,
                          1);
   }
}
```

DMA：

```
HAL_UART_Transmit_DMA(&huart1,
                      data,
                      length);

HAL_UART_Receive_DMA(&huart1,
                     buffer,
                     sizeof(buffer));
```

FreeRTOS 中建议：

```
UART 中断/DMA：只接收字节并通知任务
UART 解析任务：处理协议
UART 发送任务：统一发送，避免多个任务同时打印
```

15.4 I2C

阻塞写：

```
HAL_I2C_Mem_Write(&hi2c2,
                  device_address,
```

```
                register_address,
                I2C_MEMADD_SIZE_8BIT,
                &value,
                1,
                100);
```

阻塞读：

```
HAL_I2C_Mem_Read(&hi2c2,
                device_address,
                register_address,
                I2C_MEMADD_SIZE_8BIT,
                buffer,
                length,
                100);
```

中断和 DMA：

```
HAL_I2C_Mem_Read_IT(...);
HAL_I2C_Mem_Read_DMA(...);
HAL_I2C_Mem_Write_IT(...);
HAL_I2C_Mem_Write_DMA(...);
```

多个任务共享同一个 I2C 外设时，应该使用互斥锁：

```
osMutexAcquire(i2c_mutex, osWaitForever);
HAL_I2C_Mem_Read(...);
osMutexRelease(i2c_mutex);
```

15.5 SPI

阻塞收发：

```
HAL_SPI_TransmitReceive(&hspi1,
                        tx_data,
                        rx_data,
                        length,
                        100);
```

中断：

```
HAL_SPI_TransmitReceive_IT(&hspi1,
                           tx_data,
                           rx_data,
                           length);
```

DMA：

```
HAL_SPI_TransmitReceive_DMA(&hspi1,
                            tx_data,
                            rx_data,
                            length);
```

SPI 的 CS 通常由 GPIO 控制：

```
HAL_GPIO_WritePin(CS_GPIO_Port,
                CS_Pin,
                GPIO_PIN_RESET);

HAL_SPI_Transmit(&hspi1, data, length, 100);

HAL_GPIO_WritePin(CS_GPIO_Port,
                CS_Pin,
                GPIO_PIN_SET);
```

15.6 定时器和 PWM

启动 PWM：

```
HAL_TIM_PWM_Start(&htim2,
                  TIM_CHANNEL_1);
```

设置比较值：

```
__HAL_TIM_SET_COMPARE(&htim2,
                      TIM_CHANNEL_1,
                      compare_value);
```

启动定时器中断：

```
HAL_TIM_Base_Start_IT(&htim1);
```

回调：

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
   if (htim->Instance == TIM1)
   {
      Control_Trigger = 1;
   }
}
```

使用 FreeRTOS 后，定时器中断建议只做：

- 清除/确认硬件事件
- 读取极少量寄存器
- 发送信号量、队列或任务通知
- 设置一个事件标志

不要在中断里执行复杂 PID、阻塞 I2C、OLED 刷屏和大量浮点计算。

---

16. HAL 阻塞函数与 RTOS 任务

16.1 阻塞函数的问题

例如：

```
HAL_UART_Transmit(&huart1,
                  data,
                  length,
                  1000);
```

如果发送时间较长，当前任务会一直占用 CPU。其他低优先级任务可能无法及时执行。

16.2 推荐方式

```
任务调用 HAL 非阻塞接口启动 DMA/中断
        ↓
HAL 回调函数中发送信号量或任务通知
        ↓
任务被唤醒，继续后续处理
```

示例：

```
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    BaseType_t higher_priority_task_woken = pdFALSE;

    vTaskNotifyGiveFromISR(tx_task_handle,
                           &higher_priority_task_woken);

    portYIELD_FROM_ISR(higher_priority_task_woken);
}
```

---

17. ISR 中哪些函数可以调用

17.1 可以使用的类型

一般只使用明确标注 FromISR 的 FreeRTOS API：

```
xQueueSendFromISR()
xQueueReceiveFromISR()
xSemaphoreGiveFromISR()
vTaskNotifyGiveFromISR()
xTaskNotifyFromISR()
portYIELD_FROM_ISR()
```

CMSIS-RTOS2 中部分函数支持 ISR，但必须满足其文档限制，通常不能等待：

```
osMessageQueuePut(..., 0);
osMessageQueueGet(..., 0);
osSemaphoreRelease(...);
osThreadFlagsSet(...);
```

17.2 不应在 ISR 中调用

```
osDelay()
vTaskDelay()
osMutexAcquire()
xSemaphoreTake(..., portMAX_DELAY)
HAL_Delay()
printf()
OLED_Update()
阻塞式 HAL_I2C_Mem_Read()
```

---

18. FreeRTOS 内存管理

FreeRTOS 常见堆管理方案：

文件	特点
heap_1.c	只分配，不释放，简单
heap_2.c	可释放，但容易产生碎片
heap_4.c	合并相邻空闲块，常用
heap_5.c	支持多个不连续内存区域
heap_3.c	包装 C 标准库 malloc/free

常见配置：

```
#define configTOTAL_HEAP_SIZE      (20 * 1024)
#define configSUPPORT_DYNAMIC_ALLOCATION 1
#define configSUPPORT_STATIC_ALLOCATION  1
```

动态创建：

```
xTaskCreate(...);
xQueueCreate(...);
xSemaphoreCreateMutex();
```

静态创建：

```
StaticTask_t task_tcb;
StackType_t task_stack[256];

xTaskCreateStatic(TaskFunction,
                  "Task",
                  256,
                  NULL,
                  2,
                  task_stack,
                  &task_tcb);
```

对于控制系统，静态分配更容易预测 RAM 使用和失败情况。

---

19. FreeRTOS 配置重点

FreeRTOSConfig.h 中常见配置：

```
#define configUSE_PREEMPTION                1
#define configCPU_CLOCK_HZ                  (SystemCoreClock)
#define configTICK_RATE_HZ                  1000
#define configMAX_PRIORITIES                8
#define configMINIMAL_STACK_SIZE            128
#define configTOTAL_HEAP_SIZE               (20 * 1024)
#define configUSE_MUTEXES                   1
#define configUSE_COUNTING_SEMAPHORES       1
#define configUSE_TIMERS                    1
#define configUSE_QUEUE_SETS                0
#define configCHECK_FOR_STACK_OVERFLOW      2
#define configUSE_MALLOC_FAILED_HOOK        1
#define configGENERATE_RUN_TIME_STATS       0
```

建议打开：

```
configCHECK_FOR_STACK_OVERFLOW
configUSE_MALLOC_FAILED_HOOK
```

并实现：

```
void vApplicationStackOverflowHook(TaskHandle_t xTask,
                                   char *pcTaskName)
{
    Error_Handler();
}

void vApplicationMallocFailedHook(void)
{
    Error_Handler();
}
```

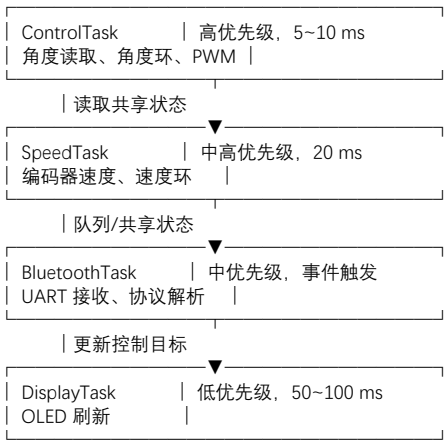
---

20. 当前平衡车项目的任务划分建议

当前项目包含：

- MPU6050
- 角度滤波
- 角度环 PID
- 速度环 PID
- 编码器
- 电机 PWM
- 蓝牙 UART
- OLED
- 按键

建议划分如下：



20.1 控制任务

角度控制任务应尽量周期稳定：

```
void ControlTask(void *argument)
{
  uint32_t last_tick = osKernelGetTickCount();

  for (;;)
  {
    MPU6050_Update();
    MPU6050_Filter();

    angle_pwm = Angle_PID(angle_target,
                  pitch_angle);

    Motor_Mix_Output(angle_pwm,
             turn_pwm);

    osDelayUntil(last_tick += 10);
  }
}
```

注意：如果 I2C 读取是阻塞式，必须确认读取时间不会超过控制周期。更理想的是使用 I2C DMA 或中断，并用信号量通知控制任务。

20.2 速度任务

```
void SpeedTask(void *argument)
{
  uint32_t last_tick = osKernelGetTickCount();

  for (;;)
```

```
    {
        speed_left = Encoder_GetSpeed(Motor_left);
        speed_right = Encoder_GetSpeed(Motor_right);

        speed_avg = 0.5f * (speed_left + speed_right);
        angle_target = Speed_PID(speed_target,
                                 speed_avg);

        osDelayUntil(last_tick += 20);
    }
}
```

20.3 蓝牙任务

推荐：

```
UART 接收中断/DMA
      ↓
字节缓冲或消息队列
      ↓
BluetoothTask
      ↓
更新 speed_target、turn_target、PID 参数
```

不要在 UART 中断中直接解析长字符串、调用 strtok()、atof() 或更新大量控制变量。

---

21. 平衡车共享数据的保护

速度任务写入：

```
speed_target
angle_target
turn_target
```

控制任务读取：

```
speed_target
angle_target
turn_target
```

如果变量是单个对齐的 int16_t、uint32_t，在 Cortex-M
上通常可以原子读写，但多个变量组成一个控制命令时，建议使用结构体加队列：

```
typedef struct
{
    float speed_target;
    float turn_target;
    float angle_kp;
    float angle_ki;
    float angle_kd;
} ControlCommand_t;
```

蓝牙任务解析完成一整条命令后，再通过队列发送给控制任务，避免控制任务读到“更新了一半”的数据。

---

22. 角度环实时性建议

平衡车角度环对实时性敏感，建议：

```
MPU6050 数据更新：5~10 ms
角度环计算：5~10 ms
速度环计算：20~40 ms
```

```
OLED 刷新：50~100 ms
蓝牙协议解析：事件触发
```

控制任务中避免：

- printf
- OLED 刷新
- 长时间 HAL 阻塞
- 动态内存申请
- strtok
- 浮点格式化输出

这些操作可以放到低优先级任务。

---

23. 选择队列、信号量、互斥锁还是任务通知

```
需要传递一条完整数据？
    └── 是 → 队列

只需要通知“事件发生”？
    └── 是 → 二值信号量或任务通知

需要表示多个资源数量？
    └── 是 → 计数信号量

需要保护共享外设？
    └── 是 → 互斥锁

需要表示多个事件状态？
    └── 是 → 事件标志组

需要一个固定周期回调？
    └── 是 → 软件定时器或周期任务

需要固定大小的数据块？
    └── 是 → 内存池
```

---

24. 常见错误

错误 1：任务中没有阻塞

错误：

```
void Task(void *argument)
{
  for (;;)
  {
    Work();
  }
}
```

如果该任务优先级较高，可能长期占用 CPU。

改进：

```
for (;;)
{
  Work();
  osDelay(1);
}
```

错误 2：在中断中调用阻塞 API

中断不能等待队列、互斥锁或延时。

错误 3：用信号量传递复杂数据

信号量只表示事件，不携带数据。复杂数据使用队列。

错误 4：用普通全局变量代替消息同步

多个任务同时修改结构体时，可能读到不完整状态。优先使用队列或互斥锁。

错误 5：任务栈过小

尤其是调用以下函数时：

- printf
- sprintf
- snprintf
- atof
- HAL 复杂外设函数
- 浮点运算

应监控任务剩余栈空间。

FreeRTOS：

```
UBaseType_t remain;

remain = uxTaskGetStackHighWaterMark(NULL);
```

错误 6：多个任务直接操作同一个 UART/OLED

应由一个专门任务统一输出，或者使用互斥锁。

错误 7：HAL 时间基准和 RTOS Tick 冲突

需要确认 SysTick 是由谁管理。STM32Cube/HAL 与 FreeRTOS 共存时，应按照工程配置选择 HAL timebase，避免 HAL_Delay() 和 RTOS Tick 互相影响。

---

25. CMSIS-RTOS2 与 FreeRTOS 原生 API 对照

功能	CMSIS-RTOS2	FreeRTOS 原生
启动内核	osKernelStart()	vTaskStartScheduler()
创建任务	osThreadNew()	xTaskCreate()
延时	osDelay()	vTaskDelay()
周期延时	osDelayUntil()	vTaskDelayUntil()
消息队列创建	osMessageQueueNew()	xQueueCreate()
发送消息	osMessageQueuePut()	xQueueSend()

功能	CMSIS-RTOS2	FreeRTOS 原生
接收消息	osMessageQueueGet()	xQueueReceive()
创建信号量	osSemaphoreNew()	xSemaphoreCreateBinary()
获取信号量	osSemaphoreAcquire()	xSemaphoreTake()
释放信号量	osSemaphoreRelease()	xSemaphoreGive()
创建互斥锁	osMutexNew()	xSemaphoreCreateMutex()
获取互斥锁	osMutexAcquire()	xSemaphoreTake()
释放互斥锁	osMutexRelease()	xSemaphoreGive()
任务通知	osThreadFlagsSet()	xTaskNotify()
事件标志	osEventFlagsSet()	xEventGroupSetBits()
软件定时器	osTimerNew()	xTimerCreate()
内存池	osMemoryPoolNew()	可用固定块分配器实现

---

26. 建议的学习顺序

1. 任务和任务状态
2. Tick、延时和优先级
3. 队列
4. 二值信号量
5. 互斥锁
6. 临界区
7. 中断与 FromISR API
8. 任务通知
9. 事件标志组
10. 软件定时器
11. 内存池和静态内存
12. 任务栈监控和系统调试

对于当前平衡车项目，建议先完成：

```
按键任务
蓝牙接收任务
OLED 显示任务
```

确认任务、队列和中断通信正常后，再把角度环和速度环迁移到高优先级控制任务。

---

27. 总结

```
HAL     负责“怎么操作硬件”
RTOS    负责“什么时候运行哪个任务”
队列    负责“传递数据”
信号量  负责“通知事件或计数资源”
互斥锁  负责“保护共享资源”
临界区  负责“保护极短的不可打断代码”
任务通知负责“快速唤醒指定任务”
```

平衡车项目中比较推荐的整体结构是：

```
UART 中断/DMA
    ↓
蓝牙任务
    ↓ 队列
控制目标
    ↓
速度任务
    ↓
目标俯仰角
```

↓
角度控制任务
↓
PWM 输出

角度环属于实时控制任务，应该保持周期稳定；蓝牙解析、OLED
显示、日志输出等功能应与控制环解耦，避免通信或显示阻塞影响小车平衡。

---

28. 参考资料

1. ARM CMSIS-RTOS2 API Reference

https://arm-software.github.io/CMSIS_5/RTOS2/html/group__CMSIS__RTOS.html

2. ARM CMSIS-RTOS2 API Function Reference

https://arm-software.github.io/CMSIS_5/RTOS2/html/rtos_api2.html

3. ST STM32Cube FreeRTOS 使用说明

https://dev.st.com/stm32cube-docs/mw-freertos/2.0.0/en/docs/markup/how_to_use.html

4. ST STM32CubeMX 用户手册

https://www.st.com/resource/en/user_manual/dm00104712-stm32cubemx-for-stm32-configuration-and-initialization-c-code-generation-stmicroelectronics.pdf